

Sea Ice Deep Fusion

Pipeline Guide: Data Flow, Scripts & Training Stages

Multimodal fusion of Sentinel-2 optical imagery and ICESat-2 ATL03 photon-altimetry

Pipeline Overview

Stage	Description	Key Scripts / Outputs
Stage 1	Raw Data Preparation	crop_all.py, crop_csv.py
Stage 2	Ground-Truth Mask Generation	segment_all.py
Stage 3	LSTM Hyperparameter Sweep	lstm_sweep.ipynb (21 configs)
Stage 4	Fusion Model Build	deep_fusion.ipynb, fusion_late_unet.ipynb, fusion_hybrid_unet.ipynb
Stage 5	Evaluation & Reproduction	test_metrics.json, confmat.png, loss_curve.png

This document describes the pipeline only: inputs, scripts, parameters, and output artifacts for each stage. Results tables, confusion matrices, and ablation study comparisons are in project_summary.pdf.

Stage 1: Raw Data Preparation

Purpose

Extract aligned 128x128 pixel patches from Sentinel-2 GeoTIFF scenes centered on each labeled ICESat-2 ground point. The patches pair optical imagery with known ground-truth positions for supervised training.

Input Data

IS2_Corrected_data/	ATL03 photon CSV files, one per beam per tile. Each row records a labeled 10-m along-track segment with pixel coordinates (pix_x, pix_y) projected into the matching Sentinel-2 tile.
----------------------------	---

S2_tiff/

Scripts

crop_csv.py	Single-pair utility. Crops all rows in one CSV from one TIFF. Writes PNGs to outputs/{csv_stem}__{tif_stem}/ together with a manifest.csv tracing each patch back to its CSV row, pixel coordinates, and class label.
--------------------	---

crop_all.py

Key Parameters

Patch size	128 x 128 pixels
-------------------	------------------

Center align

Command

```
python crop_all.py          # batch: all 6 CSV/TIFF pairs -> outputs/  
python crop_csv.py         # single-pair diagnostic run
```

Output

- outputs/ -- flat directory of PNG patches, one per labeled CSV row
- Each patch: 128x128 RGB, uint8, labeled point at center

Stage 2: Ground-Truth Mask Generation

Purpose

Convert each RGB patch into a three-class per-pixel segmentation mask using an HSV color-thresholding pipeline with shadow and cloud removal. The masks serve as the ground truth for all classifier training and evaluation.

Script

segment_all.py	Applies shadow_cloud_removal() then color_segmentation() to every PNG in outputs/ and writes the result to outputs_segmented/ under the same filename. Skips files that already have a segmented counterpart so the script is safely resumable.
-----------------------	---

Algorithm

- Step 1 - Open-water isolation: mask pixels where HSV V <= 30 as water; replace with white to prevent influence on subsequent

steps.

- Step 2 - Shadow/cloud removal: grayscale dilate + median blur (kernel 155) builds a background model; absdiff + Otsu threshold isolates foreground; normalize to 0-255, truncate highlights at 235.
- Step 3 - Color segmentation: re-classify the cleaned image via HSV thresholds into three classes. Thick ice: V in [205, 255]. Thin ice: V in [31, 204]. Open water: V in [0, 30].
- Step 4 - Pixel labeling: thick ice -> red (255,0,0), thin ice -> blue (0,0,255), open water -> green (0,255,0).

HSV Thresholds

Thick ice	H [0,185], S [0,255], V [205,255] -> red (255, 0, 0)
------------------	--

Thin ice

Command

```
python segment_all.py      # inputs: outputs/    outputs: outputs_segmented/
```

Output

- outputs_segmented/ -- 3-class RGB masks, same filenames as input patches
- Not version-controlled; regenerate from outputs/ as needed

Stage 3: LSTM Hyperparameter Sweep

Purpose

Train a standalone bidirectional LSTM (BiLSTM) photon-branch classifier across 21 configurations to select the hyperparameters that maximize mean IoU on a held-out validation split. The winner checkpoint is transferred to Stage 4.

Script

lstm_sweep.ipynb	Runs all 21 configs sequentially. Each config trains to convergence, evaluates on the held-out tile T03CWT, and writes per-config artifacts to archive/runs/lstm_sweep/{config_name}/. Aggregated results are in archive/runs/lstm_sweep/sweep_results.csv.
-------------------------	---

Input Feature Construction

ATL03 photon records are aggregated to 10-meter along-track segments. A sliding window of consecutive segments is centered on each labeled point; 8 scalar features are extracted per segment:

h_cor_mean	Mean corrected photon height (m)
-------------------	----------------------------------

h_cor_mean

All 21 Sweep Configurations

One parameter is varied at a time. The baseline uses the italicized default. Bold rows indicate the winning configuration.

Config	Focal alpha	gamma	hidden	lr	dropout	seq	seed
baseline	[0.05,0.45,0.60]	2.0	48	8.886e-4	0.4	5	42
alpha [0.02,0.44,0.54]	[0.02,0.44,0.54]	2.0	48	8.886e-4	0.4	5	42
alpha [0.05,0.45,0.50]	[0.05,0.45,0.50]	2.0	48	8.886e-4	0.4	5	42
alpha [0.05,0.50,0.45]	[0.05,0.50,0.45]	2.0	48	8.886e-4	0.4	5	42
alpha [0.041,0.409,0.55]	[0.041,0.409,0.55]	2.0	48	8.886e-4	0.4	5	42
gamma 1.0	[0.05,0.45,0.60]	1.0	48	8.886e-4	0.4	5	42
gamma 3.0	[0.05,0.45,0.60]	3.0	48	8.886e-4	0.4	5	42
gamma 5.0	[0.05,0.45,0.60]	5.0	48	8.886e-4	0.4	5	42
hidden 32	[0.05,0.45,0.60]	2.0	32	8.886e-4	0.4	5	42
hidden 64	[0.05,0.45,0.60]	2.0	64	8.886e-4	0.4	5	42
hidden 96 *WINNER*	[0.05,0.45,0.60]	2.0	96	8.886e-4	0.4	5	42
lr 5e-4	[0.05,0.45,0.60]	2.0	48	5.0e-4	0.4	5	42
lr 1e-3	[0.05,0.45,0.60]	2.0	48	1.0e-3	0.4	5	42
lr 2e-3	[0.05,0.45,0.60]	2.0	48	2.0e-3	0.4	5	42
dropout 0.2	[0.05,0.45,0.60]	2.0	48	8.886e-4	0.2	5	42
dropout 0.3	[0.05,0.45,0.60]	2.0	48	8.886e-4	0.3	5	42
dropout 0.5	[0.05,0.45,0.60]	2.0	48	8.886e-4	0.5	5	42
seq_len 3	[0.05,0.45,0.60]	2.0	48	8.886e-4	0.4	3	42
seq_len 7	[0.05,0.45,0.60]	2.0	48	8.886e-4	0.4	7	42

seed 7	[0.05,0.45,0.60]	2.0	48	8.886e-4	0.4	5	7
seed 123	[0.05,0.45,0.60]	2.0	48	8.886e-4	0.4	5	123

Winner Configuration

hidden_dim	96
------------	----

focal alpha

Winner Artifacts

- runs/bilstm/test_metrics.json -- per-class IoU, mIoU, macro F1
- runs/bilstm/confmat.png -- row-normalized confusion matrix
- runs/bilstm/metrics.csv -- epoch-level validation metrics
- archive/runs/lstm_sweep/sweep_results.csv -- all 21 configs compared

Stage 4: Fusion Model Build

Purpose

Combine the pretrained U-Net optical branch and the BiLSTM photon branch into a joint multimodal classifier. Three fusion strategies are implemented: Deep Fusion (primary result), Late Fusion, and Hybrid Fusion. Deep Fusion fuses the modalities entirely at the feature level and achieves the best test performance.

Input

outputs/ 128x128 RGB patches from Stage 1

outputs_s

Fusion Scripts

deep_fusion.ipynb Primary result. Fuses modalities at deep feature level via channel projection, spatial broadcast, and Squeeze-and-Excitation recalibration.

fusion_late

Deep Fusion Build Sequence

- Load pretrained BiLSTM checkpoint (Stage 3 winner) into photon branch.
- Initialize U-Net encoder (ResNet-18, ImageNet pretrained) via segmentation-models-pytorch. Decoder output: 16-channel feature map (16, 128, 128).
- Fusion bridge: project photon embedding (hidden_dim -> 16 channels) via 1x1 conv, broadcast the per-point vector spatially to fill the 128x128 grid.
- Concatenate broadcast photon features with U-Net decoder map -> 32-channel tensor.
- Squeeze-and-Excitation (SE) block recalibrates channel weights: global average pool -> FC(32->8) -> ReLU -> FC(8->32) -> Sigmoid -> channel-wise scale.
- 1x1 convolution maps the 32 recalibrated channels to 3 class logits.
- Train 30 epochs; Adam optimizer; base learning rate for U-Net head; photon branch fine-tuned at 0.1x base learning rate.

Learning Rate Rationale

The photon branch enters fusion training with representations already optimized for the standalone classification task. A 10x lower learning rate (0.1x base) allows the branch to adapt to the joint fusion context -- adjusting its features to complement the U-Net -- while retaining the general photon-discrimination patterns learned in Stage 3. At full learning rate the pretrained weights are overwritten too quickly; at zero (frozen) the branch cannot specialize to the fusion setting. The 0.1x ratio was validated against both extremes in the ablation variants (fusion_v2 through fusion_v4 in archive/).

Output Artifacts

runs/deep_fusion/test_metrics.json Per-class IoU (thick, thin, water), mIoU, macro F1

runs/deep

Reproduction Commands

```
# Deep Fusion (primary result)
```

```
jupyter nbconvert --to notebook --execute deep_fusion.ipynb

# Late Fusion
jupyter nbconvert --to notebook --execute fusion_late_unet.ipynb

# Hybrid Fusion
jupyter nbconvert --to notebook --execute fusion_hybrid_unet.ipynb

# Note: set CUDA_VISIBLE_DEVICES=0 before launching.
# Each 30-epoch run requires ~60-90 min on an RTX A6000.
```

Stage 5: Evaluation Metrics & Full Reproduction

Evaluation Metrics

All models are evaluated on the geographically held-out tile T03CWT. No patches from T03CWT appear in any training or validation split.

mIoU	Mean Intersection-over-Union across the three classes. Primary ranking metric.
-------------	--

Per-class I

Full Pipeline Reproduction

```
# Stage 1: extract 128x128 patches
python crop_all.py

# Stage 2: generate 3-class ground-truth masks
python segment_all.py

# Stage 3: LSTM hyperparameter sweep (21 configs)
jupyter nbconvert --to notebook --execute lstm_sweep.ipynb

# Stage 4a: train Deep Fusion model (primary result)
jupyter nbconvert --to notebook --execute deep_fusion.ipynb

# Stage 4b: train Late Fusion model
jupyter nbconvert --to notebook --execute fusion_late_unet.ipynb

# Stage 4c: train Hybrid Fusion model
jupyter nbconvert --to notebook --execute fusion_hybrid_unet.ipynb
```

Directory Reference

IS2_Corrected_data/	ICESat-2 ATL03 photon CSVs (input). 6 files: 2 beams x 3 tiles.
----------------------------	---

S2_tiff/

Environment

Python

3.9 or later

GPU**Notes**

- Large datasets, model checkpoints (*.pt), and patch caches are excluded from version control via .gitignore and must be regenerated or supplied locally.
- Notebooks define input/output paths in a configuration cell near the top; adjust these to match your local directory layout before running.
- The notebook_output/ directory contains the ATL03 LSTM data preparation notebook used to build the photon feature sequences.